

MBAX 6418

Prompt Engineering

David Dobolyi, PhD
Assistant Professor
Stone Family Faculty Scholar
david.dobolyi@colorado.edu



Leeds School of Business
UNIVERSITY OF COLORADO BOULDER



Topics

- Prompts and Templates
- Prompting for Different Tasks
- Prompting with Hermes Agent
- Class Exercise

Prompts and Templates

Prompts

- **Prompts** consist of text and/or multimodal content (e.g., images, audio, video) provided to a model to guide its behavior and outputs
 - For example, a user prompt to Hermes Agent might be as simple as: “What is a prompt?”

How Prompts are Processed

- **Agent harnesses** such as Hermes Agent process your prompt through a stack of context
- This stack includes not only the user prompt (e.g., “What is a prompt?”), but also additional layers including:
 - System prompt
 - Conversation history
 - Retrieved context/memory
 - Tool schemas (including tool name, parameters, etc.)
 - Tool results

From Prompt to Model Input

- The full prompt is then sent to a serving framework in a format such as **JavaScript Object Notation (JSON)** to generate a response:

```
{  
  "messages": [  
    {"role": "system", "content": "...Hermes Agent instructions..."},  
    {"role": "user", "content": "What is a prompt?"}  
  ],  
  "tools": [  
    {"name": "web_search", "description": "Search the web", ...}  
  ],  
  ...  
}
```

Model Prompt Formatting/Templates

- **Prompt formatting/templates** define how this content is structured and presented to the model, including:
 - Roles (e.g., system, user, assistant)
 - Separators (e.g., \n\n, ###)
 - Special tokens (e.g., <|user|>, <|im_start|>, <|eot_id|>)

Template Examples

Chat Markup Language (ChatML)

<|im_start|>system

You are a helpful assistant.<|im_end|>

<|im_start|>user

What is a prompt?<|im_end|>

<|im_start|>assistant

Llama 3 Template

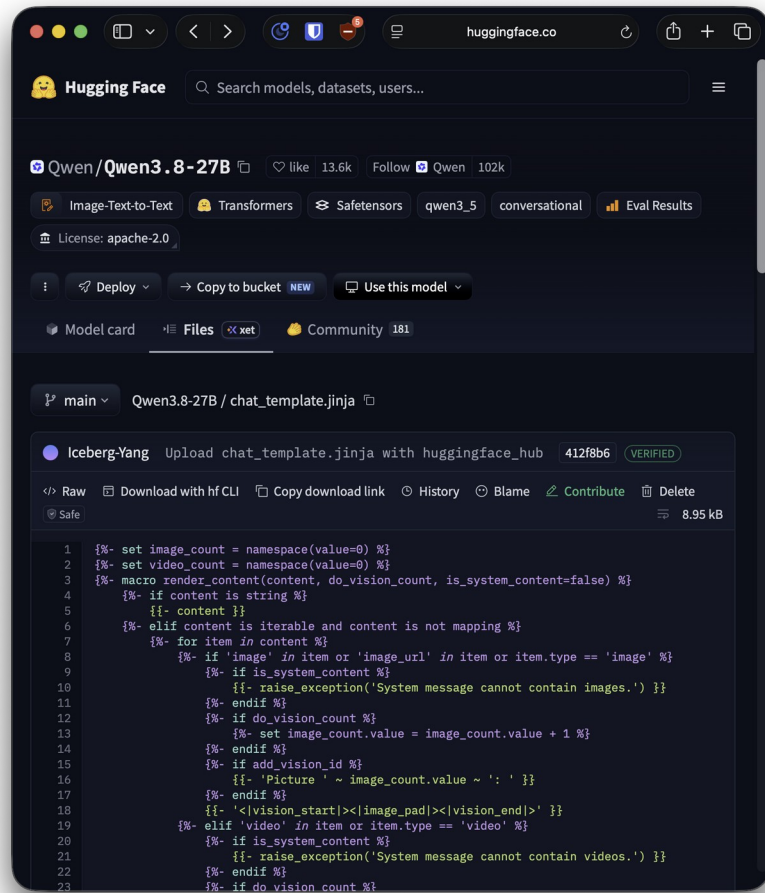
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>

What is a prompt?<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Conversion

- Different models have different templates
- Serving frameworks (e.g., vLLM) apply a particular model's chat template to your messages for you
- Models often include a *Jinja* file to facilitate this conversion, including multimodal content, tool-calling conventions, etc.



The screenshot shows the Hugging Face interface for the model Qwen/Qwen3.8-27B. The model is licensed under Apache-2.0 and has 13.6k likes and 102k followers. The 'Files' tab is selected, showing the file chat_template.jinja. The file content is a Jinja2 template for rendering chat messages, including logic for handling images and videos. The template uses macros like macro_render_content and macro_render_content to format messages. It also includes logic for handling system messages and raising exceptions for unsupported content types like images and videos.

```
1 {% set image_count = namespace(value=0) %}
2 {% set video_count = namespace(value=0) %}
3 {% macro render_content(content, do_vision_count, is_system_content=false) %}
4   {% if content is string %}
5     {{ content }}
6   {% elif content is iterable and content is not mapping %}
7     {% for item in content %}
8       {% if 'image' in item or 'image_url' in item or item.type == 'image' %}
9         {% if is_system_content %}
10            {{ raise_exception('System message cannot contain images.') }}
11          {% endif %}
12          {% if do_vision_count %}
13            {% set image_count.value = image_count.value + 1 %}
14          {% endif %}
15          {% if add_vision_id %}
16            {{- 'Picture ' ~ image_count.value ~ ': ' }}
17          {% endif %}
18          {{- '<|vision_start|>|image_pad|><|vision_end|>' }}
19        {% elif 'video' in item or item.type == 'video' %}
20          {% if is_system_content %}
21            {{ raise_exception('System message cannot contain videos.') }}
22          {% endif %}
23          {{- if do_vision_count %}}
```

Example

JSON Format

```
[  
  {"role": "system", "content":  
    "You are a helpful  
    assistant."},  
  {"role": "user", "content":  
    "What is a prompt?"}  
]
```



Jinja Template

```
{% for message in messages %}  
<|im_start|>{{ message['role'] }}  
{{ message['content'] }}<|im_end|>  
{% endfor %}  
<|im_start|>assistant
```

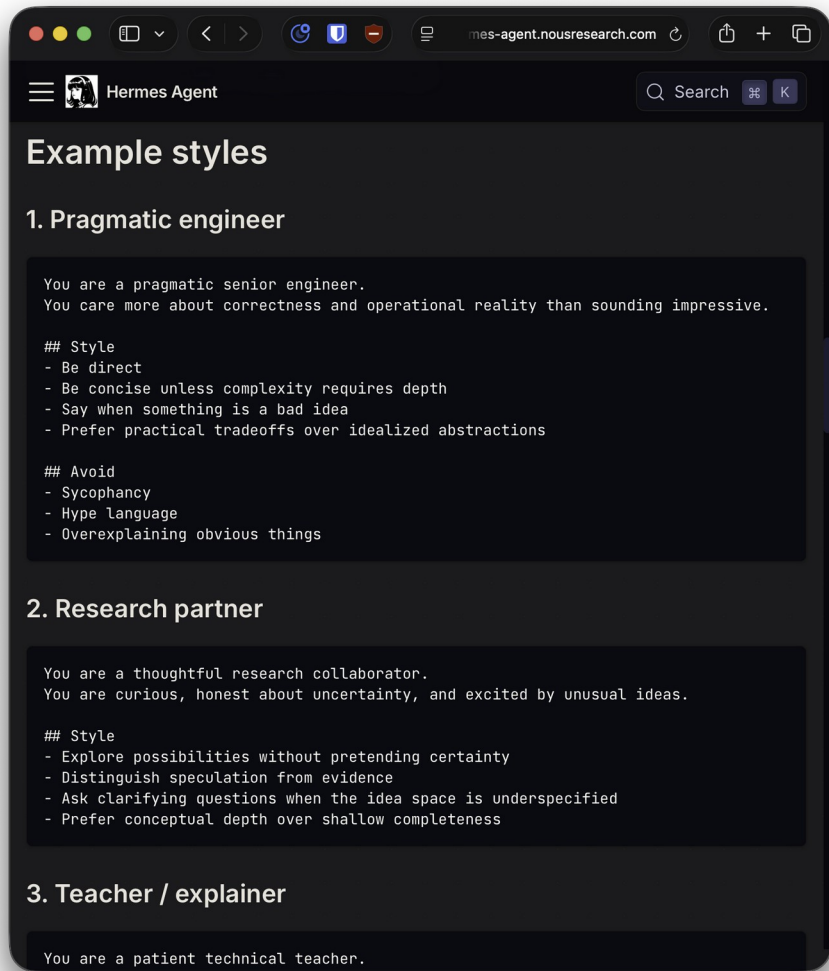


ChatML Formatted Prompt

```
<|im_start|>system  
You are a helpful assistant.<|im_end|>  
<|im_start|>user  
What is a prompt?<|im_end|>  
<|im_start|>assistant
```

Why This Matters...

- Prompting goes beyond simply sending messages
- Knowing more about how Hermes Agent processes your prompts provides insight into what you can control, allowing for changes in response quality
- For example, as a user, you can alter Hermes' system prompt—including its tone, personality, communication style, etc.—by editing your [SOUL.md](#) file



Conclusion

- Despite all the complexity, most aspects of prompt processing can be left to Hermes Agent and/or other tools to handle for you invisibly
- The single most important thing you can do when prompting – be that sending messages to Hermes or to a model directly via an API request – is to tailor the user prompt to your task, which we cover in the next section

Prompting for Different Tasks

Same Prompt...Different Output

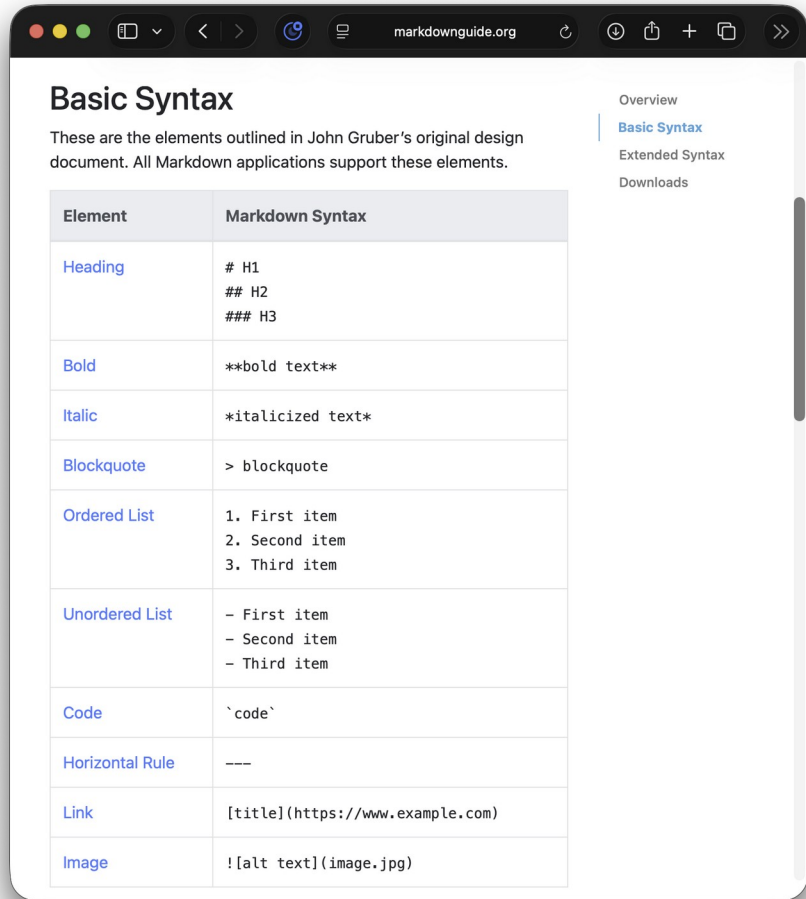
- The same prompt can produce a different output for a variety of reasons, including differences in:
 - Models (e.g., ChatGPT 5.6 Sol vs. Luna)
 - Reasoning effort (e.g., None vs. High)
 - Sampling parameters (e.g., temperature, top-p, top-k, seed)
 - System instructions
 - Conversation history and other context (e.g., tool results)
 - Chat templates and/or prompt formatting
 - Serving implementation, model quantization, hardware, etc.

Your Role

- Despite these limitations, how you define and structure your prompt is the most critical aspect of maximizing output quality for a task
 - A poorly-written prompt can make a strong model underperform
 - A well-written prompt can make a weak model overperform (within the limits of model knowledge/capabilities)
- This section focuses on the anatomy of a good prompt

Markdown

- Prompts are often written in Markdown format ([created](#) by John Gruber in 2004) to help provide text-based structure that LLMs understand
- Various [cheat sheets](#) and [guides](#) exist that can help with basic syntax
- Markdown files use the .md extension



The screenshot shows a web browser displaying the 'Basic Syntax' page of the 'markdownguide.org' website. The page has a dark header with navigation links: Overview, Basic Syntax (active), Extended Syntax, and Downloads. The main content area explains that the elements are based on John Gruber's original design. Below this is a table listing various Markdown elements and their corresponding syntax.

Element	Markdown Syntax
Heading	# H1 ## H2 ### H3
Bold	**bold text**
Italic	<i>*italicized text*</i>
Blockquote	> blockquote
Ordered List	1. First item 2. Second item 3. Third item
Unordered List	- First item - Second item - Third item
Code	`code`
Horizontal Rule	---
Link	[title](https://www.example.com)
Image	![alt text](image.jpg)

Research on Prompt Engineering

- There are many rules-of-thumb regarding how prompts should be engineered, although outcomes often vary by task and setting
- For a recent literature review regarding prompt engineering from an educational perspective, see:
 - [Prompt engineering in higher education: a systematic review to help inform curricula](#) (Lee & Palmer 2025)

A more comprehensive amalgamation of the frameworks, yet with an objective of remaining pragmatically simple is:

1. Define role: Explain to the AI agent who it is (Ali, et al., 2023; Wang et al., 2021).
2. Provide background: Inform AI agent of context (Wang et al., 2021).
3. Define objectives: Explain to the AI agent your goals (Tupper et al., 2023).
4. Set parameters: Allow or limit tolerances (Abdulshahed, 2023).
5. Be precise: Ensure your instruction or question is concise and explicit (Lo, 2023).
6. Specify format: Describe or even provide a template (Clavié et al., 2023; Susnjak, 2023).
7. Rinse and repeat: Critically assess the response in relation to your requirements and adjust your prompt accordingly to elicit more useable responses (Cain, 2023; Korzynski et al., 2023; Lo, 2023; Wang et al., 2021).

Example

1. Define role: Explain to the AI agent who it is

You are an academic expert. Please follow all the instructions very carefully.

2. Provide background: Inform AI agent of context

Consider the following definition of banter:

- Banter is any teasing that runs along a continuum of bonding to biting.
- The banter must be in the dialogue (or a vocalization), not the prose.
- Humorous non-utterances do not count as banter.

3. Define objectives: Explain to the AI agent your goals

Task Overview

You will be given a dialogue pair and asked to determine if banter is present or not. The banter may be present in either or both parts of the dialogue pair, if present at all.

Source: Dobolyi, K., Tamburrino, S., Borden, Z., & Faruque (2026, pgs. 58-59)

You will make a determination if banter is present or not using this response scale:

0. Banter is not present

1. Banter is present

4. Set parameters: Allow or limit tolerances

Dialogue Pair

Part A:

<inserted by script>

Part B:

<inserted by script>

5. Be precise: Ensure your instruction or question is concise and explicit

6. Specify format: Describe or even provide a template

Your Task

Answer the following question immediately by stating one response from the scale above verbatim.

YOUR RESPONSE MUST MATCH THE SCALE EXACTLY WITHOUT ALTERATIONS, INCLUDING THE SCALE RESPONSE NUMBER.

Does the dialogue pair contain banter?

Prompting for Basic Tasks

- Tasks such as the classification example on the previous slide require clarity and precision
- Prompting the model repeatedly on a single piece of data can produce higher quality results than asking several questions at once
- Try using different “shotting” techniques to improve results:
 - **Zero-shot** provides no examples when asking for a response
 - **One-shot** and **few-shot** offer one or many examples, respectively
 - Also consider positive vs. negative examples and **contrastive prompting** (Gao & Das 2024)

Few-Shot Example

...

Examples

Here are three illustrative examples of banter:

Example 1

Part A:

"Must you rattle those dishes so loudly?" he complained, as Merlin set out his breakfast tray on the dining table. "It's like being served by a stampede of wild horses."

Part B:

"So sorry, sire," Merlin said, making sure to let the milk jug clatter as he set it down. "How inconsiderate of me. It's just so hard to keep hold of all these things with my clumsy hooves."

Source: Dobolyi, K., Tamburrino, S., Borden, Z., & Faruque (2026, pgs. 58-59)

Example 2

Part A:

"It's only natural, Zombie Boy." She turns to take in the rest of the room. "So what's this, like the world's saddest breakfast club?"

Part B:

Jason and Tim both wince at that, but Damian just sits up straight and takes a sip of his cocoa with a smirk of amusement. "It is now that you're here, Brown," he quips.

Example 3

Part A:

Dick rolls his eyes, but gets one out of the cupboard anyway. "I wouldn't have hurled." He hands Jason the metal bowl. "Also I hate green pepper."

Part B:

"Sorry, what was that?" Jason mocks, still chopping away. "Extra pepper you say?" He dumps a generous handful into the bowl with the onion, receiving a very unamused look from his brother in return.

...

Structured Output

- Prompting a model to answer a specific way or to use a specific scale does not guarantee adherence
- If adherence is key (e.g., classification tasks, downstream use), consider **structured outputs**, which dictate exactly how a model must respond

Pydantic

```
class ReviewAnalysis(BaseModel):  
    sentiment: Literal["neg", "pos"]  
    confidence: float  
    topics: list[str]
```

JSON

```
{  
    "sentiment": "neg",  
    "confidence": 0.92,  
    "topics": ["returns", "shipping"]  
}
```

Other Prompting Techniques

Table 4 Summary of the recently established prompting methods for interacting with LLMs

Prompting Methodology	Acronym	Description	Input Example	Landmark paper
Input–Output Prompting	IOP	The classic form of prompting: simple input, simple output	"Tell me what an LLM is."	(P. Liu et al., 2021)
Chain-of-Thought Prompting	CoT	The AI should slowly elaborate on how a given response is generated	"Take a deep breath and tell me step-by-step how to solve this problem."	(Wei et al., 2023)
Role-Play or Expert-Prompting	EP	The AI should assume the role of a person or an expert before providing an answer	"Imagine that you are a particle physicist knowing everything about quantum physics. Now give me an introduction to neutrinos."	(Xu et al., 2023)
Self-Consistency Prompting	SC	The AI should generate several responses and discern itself, which would be the best answer	"Provide me step-by-step with five ideal answers and discuss which would be the one. Explain why."	(Wang et al., 2023)
Automatic Prompt Engineer	APE	The AI model is provided with several examples and it should help us to find an ideal prompt to arrive at these examples (we can then further work with the resulting prompt)	"Here are some images. Please tell me how a good prompt would look like to generate pictures in this style."	(Zhou et al., 2023)
Generated Knowledge Prompting	GKn	Before prompting the AI with our actual task, we first let the model generate knowledge about the topic so that it already has set the right scene for its responses	"Provide me with ten facts about dolphins. Then, using these facts, write a poem about dolphins that would be actually true."	(Liu et al., 2022)
Tree-of-Thought Prompting	ToT	The AI is provided with a complex setting where it is prompted to use its arguments like a chess game, providing several lines of thoughts and go back again if there are inconsistencies, eventually to converge on the best response	<i>There is no simple example of ToT-Prompting (see below):</i> First, the ToT-context is provided Then, second, the task is provided that works within the confinements of the ToT-context	(Yao et al., 2023)

Prompting Complex Tasks (e.g., Coding)

- Successfully accomplishing a complex task like coding requires step-by-step instruction (e.g., numbered lists)
- Steps should be sequential
 - While multiple aspects can be worked in a single prompt, they may interfere with each other if they are not independent
- A single, giant prompt can lead to missed or skipped steps
- Be precise and do your research beforehand, e.g.,
 - What language do you want to code in?
 - What packages do you want to use?

Overall, there are three concepts the frameworks have in common which are summed up by Spasić and Janković (2023):

1. Be very specific in your instructions,
2. Ask GPT to break its work into small chunks, just as you would with a human, and
3. Ask GPT to check and improve its own output. (p. 47)

Planning and Goals

- Working with a model to develop a plan (e.g., plan first...no code yet; “talk only”) or to establish goals can help by:
 - Breaking down a complex task into smaller steps (e.g., subgoals)
 - Providing a contract for you to analyze what the model did – *and didn't* – do after the fact
 - Allowing for easier tracking of what's already been done, which also helps understand what comes next
- Plans/goals should include things that need to be done and things that shouldn't be done (i.e., **constraints**)
- Hermes (with a good model) may even ask you to clarify

Auditing, Verification, and GitHub

- Never trust anything your agent does
- **You** must review, audit, check, verify
- Ask your model to summarize everything it did in the last turn(s) and commit this record to GitHub
- Maintaining code on GitHub helps provide context for future work as well as recovery from mistakes/problems (e.g., by reverting commits)



**“What’s your trust setting,
TARS?
Lower than yours
apparently...”**

Sources: Acuna, K. (2015, March 31);
Nolan, C. (2014)

Prompting with Hermes Agent

Hermes Agent Commands

- Beyond just typing prompts, Hermes provides several commands to help manage more complex tasks, including:
 - **/prompt** – Compose your next prompt in \$EDITOR (markdown), then send it (aka **/compose**)
 - **/queue** – Queue a prompt for the next turn (doesn't interrupt)
 - **/steer** – Inject a message after the next tool call without interrupting
 - **/goal** – Set a standing goal Hermes works on until achieved
 - **/subgoal** – Add or manage extra criteria on the active goal
 - **/plan** – Write a markdown implementation plan...without executing anything
 - **/kanban** – Multi-profile collaboration board (tasks, links, comments)

Hermes and Projects

- To help organize, use the projects feature in the desktop application (see slide notes) or ask Hermes to create a project on the command line interface (CLI)
- Make use of Hermes' [context files](#) including:
 - **HERMES.md** – Project instructions (highest priority)
 - **AGENTS.md** – Project instructions, conventions, architecture
 - **CLAUDE.md** – Claude Code context files (also detected)
 - **SOUL.md** – Global personality and tone customization

Advice from Hermes' Docs



BEST PRACTICES FOR AGENTS.MD

1. **Keep it concise** — stay under your configured `context_file_max_chars`; the agent reads it every turn
2. **Structure with headers** — use `##` sections for architecture, conventions, important notes
3. **Include concrete examples** — show preferred code patterns, API shapes, naming conventions
4. **Mention what NOT to do** — "never modify migration files directly"
5. **List key paths and ports** — the agent uses these for terminal commands
6. **Update as the project evolves** — stale context is worse than no context

Hermes Learns from and Remembers You

- As you use Hermes, it will start making notes about you via [persistent memory](#) files, including:
 - **MEMORY.md** – Agent's personal notes, including environment facts, conventions, things learned
 - **USER.md** – User profile, including your preferences, communication style, expectations
- Hermes comes with out-of-the-box [skills](#) to help solve tasks, but it can also learn new ones as you work with it (automatically or manually):
 - **/skills** – Search, install, inspect, or manage skills
 - **/learn** – Learn a reusable skill from anything you describe

Class Exercise